

CSE30301 Homework 1

Statistical Decision-Making Under Uncertainty

Youngsoo Jang (youngsoo.jang@unist.ac.kr)

Due Date: May 14, 2026, 11:59 PM

Submission: Blackboard

1. Introduction

In previous lectures, we studied key ideas from probability and statistics, including random variables, expectation, variance, sample mean, confidence intervals, uncertainty estimation, and model selection based on observed data. These tools are useful not only for analyzing static datasets, but also for making sequential decisions under uncertainty. In many real-world problems, we must repeatedly choose an action while outcomes are uncertain. Examples include selecting advertisements, recommending products, choosing treatments, or testing robotic strategies. In this assignment, you will design your own statistical decision-making strategy for a simple sequential decision-making problem.

2. Project Instruction

2.1. Multi-Armed Bandit Problem

A classic framework for such problems is the multi-armed bandit problem. Imagine several slot machines in front of you. Each machine gives random rewards, but some machines are better than others. You do not know which one is best in advance. Your task is to repeatedly choose machines while learning from past observations. This creates a fundamental tradeoff:

- **Exploration:** try uncertain options to gather information.
- **Exploitation:** choose the option that currently looks best.

Balancing these two goals is the core challenge of this assignment.

2.2. Objective of This Assignment

In this assignment, we consider a stochastic multi-armed bandit problem with K possible actions, called **arms**. At each round $t = 1, 2, \dots, T$:

1. the learner selects one arm $a_t \in \{1, 2, \dots, K\}$,
2. the environment returns a random reward r_t ,
3. the learner uses the observed reward to improve future decisions.

The process is repeated for a total of T rounds, where T is called the **horizon**.

Arm. An **arm** represents one available option or action. Examples include selecting one advertisement, recommending one product, or trying one strategy.

Reward. Each arm i produces random rewards drawn from an unknown probability distribution with mean:

$$\mu_i = \mathbb{E}[r \mid a = i]$$

The quantity μ_i is the expected reward of arm i . A higher μ_i means that the arm is better on average.

Best Arm. The optimal expected reward is:

$$\mu^* = \max_{i \in \{1, \dots, K\}} \mu_i$$

Any arm achieving μ^* is called an best arm.

Regret. Since the learner does not know the best arm in advance, it may choose suboptimal arms during learning. The instantaneous regret at round t is:

$$\mu^* - \mu_{a_t}$$

which measures how much reward is lost by choosing arm a_t instead of the best possible arm. The cumulative regret after T rounds is:

$$\text{Regret}(T) = \sum_{t=1}^T (\mu^* - \mu_{a_t})$$

A smaller regret means the algorithm quickly identified strong arms and avoided poor decisions.

Your Goal. Design an algorithm that uses past observations to choose future actions so that cumulative regret becomes as small as possible.

Lower Regret = Better Strategy

3. Example Baseline Algorithms

The provided code includes three simple baseline algorithms. These methods illustrate different ways of balancing **exploration** and **exploitation**. Let $\hat{\mu}_i(t)$ denote the current sample mean reward of arm i estimated from observations up to round t .

$$\hat{\mu}_i(t) = \frac{1}{N_i(t)} \sum_{s \leq t: a_s = i} r_s$$

where $N_i(t)$ is the number of times arm i has been selected so far.

3.1. Greedy

The Greedy algorithm always selects the arm with the highest current estimated mean reward:

$$a_t = \arg \max_i \hat{\mu}_i(t)$$

This method performs pure exploitation by always trusting the current best estimate, but it may fail to explore arms that were underestimated early by chance.

3.2. Epsilon-Greedy

The ϵ -Greedy algorithm explores randomly with probability ϵ , and behaves greedily with probability $1 - \epsilon$:

$$a_t = \begin{cases} \text{random arm,} & \text{with probability } \epsilon \\ \arg \max_i \hat{\mu}_i(t), & \text{with probability } 1 - \epsilon \end{cases}$$

This method introduces explicit random exploration, which helps avoid premature commitment, although some exploration steps may be inefficient.

3.3. Softmax Action Selection

Softmax converts estimated rewards into probabilities:

$$p_i(t) = \frac{\exp(\hat{\mu}_i(t)/\tau)}{\sum_{j=1}^K \exp(\hat{\mu}_j(t)/\tau)},$$

where $\tau > 0$ is the temperature parameter. Then the algorithm samples:

$$a_t \sim \text{Categorical}(p_1(t), \dots, p_K(t))$$

This method explores more promising arms more frequently while still assigning some probability to weaker arms, and the exploration level is controlled by τ .

4. Code Structure

You are given the following files. Understanding their roles will help you navigate the assignment more efficiently.

- `bandit_core.py`: contains the core bandit environment, reward generation process, regret computation, and utility functions used during evaluation.
- `algorithms_baseline.py`: provides implementations of several example baseline algorithms such as Greedy, ϵ -Greedy, and Softmax. You may use these implementations as references.
- `algorithms_student.py`: contains the `student_algorithm(...)` function. This is the main file you must modify and submit.
- `evaluate_public.py`: runs your algorithm on the released public environments and reports performance metrics such as cumulative regret.
- `plot_regret.py`: generates visualization plots of regret curves on the public environments, which may help you compare algorithms and debug behavior.

5. Your Programming Task

In this assignment, your main task is to implement your own bandit strategy by modifying the function `student_algorithm(...)` inside the file `algorithms_student.py`. This function will be repeatedly called during interaction with the environment, and at each round, it must decide which arm to choose based on the information observed so far. You are free to design any strategy you would like, including methods based on sample means, confidence bounds, randomized exploration, adaptive schedules, or your own original ideas.

Important: Function Signature Must Not Change

Please keep the provided function name, input arguments, and output format exactly the same as in the starter code. **Do not rename the function or modify its interface.** The autograder will directly call this function, so changing the signature may cause your submission to fail.

6. Submission Instructions

Please submit your result as one compressed file through Blackboard. Create a single zip file named `{student_ID}.zip` where `student_ID` should be replaced by your actual student ID number. This zip file must contain the following two items.

6.1. Python source file

First, submit your Python source file containing your implemented algorithm. Rename the file `algorithms_student.py`

to

`algorithms_{student_ID}.py`

where `student_ID` should be replaced by your actual student ID number. For example,

`algorithms_20251234.py`

This naming rule is important for efficient grading, so please follow it exactly.

6.2. Report

Second, submit a short report in PDF format (file name: `report_{student_ID}.pdf`). A recommended length is 2-3 pages. The report should briefly explain your algorithm with mathematical equations, the intuition behind your design, why you expect it to perform well, and any experiments or observations you found while testing your method. The report should clearly include the following items:

- What intuition motivated your algorithm design.
- A mathematical description of your method using formulas or pseudocode.
- Whether you created and tested additional environments beyond the provided public environments in order to search for a stronger algorithm.
- If your method uses coefficient terms or hyperparameters, specify their final values and explain how those values were selected.
- Numerical results and plots for regret on the provided public environments.

7. Evaluation

Your final grade will be based on the following three components.

Performance on Public Environment (20%)

First, your algorithm will be evaluated on the provided public environments using the released evaluation code. Performance will be measured by cumulative regret, where lower regret indicates better decision-making.

Performance on Hidden Environment (50%)

Second, your submitted algorithm will be tested on additional hidden environments that are not released in advance. This is intended to evaluate robustness and generalization beyond the public test cases.

Report (30%)

Finally, your written report will be graded based on clarity, quality of explanation, motivation of the method, and overall reasoning behind your algorithm design.

8. Use of AI Tools

The use of AI tools is permitted for this assignment. This includes, but is not limited to, using AI assistants for background research, coding support, debugging help, writing assistance, and organizing your report. However, simply relying on AI-generated outputs without personal understanding is strongly discouraged. The core purpose of this assignment is for you to think carefully about the problem of sequential decision making under uncertainty and to develop your own reasoning about what makes an effective algorithm. You are expected to make the key design decisions yourself, critically evaluate any suggestions produced by AI tools, and ensure that your final submission reflects your own understanding and effort. In particular, your report should contain evidence of your own thought process, experimentation, and reasoning rather than a copied or generic explanation. AI tools may assist your work, but they should not replace your thinking.

9. Final Note

You are not expected to know advanced bandit algorithms beforehand. This assignment is designed for you to apply probability and statistics ideas learned in class to a new decision-making setting. Use logic, intuition, and experimentation.

A. Appendix: Environment Setup

A.1. Python Environment Setup

If Python is not already installed on your computer, we strongly recommend using Anaconda, which provides Python together with commonly used scientific packages such as NumPy. Anaconda can be downloaded from:

`https://www.anaconda.com/products/distribution`

After installing Anaconda, open a terminal (or Anaconda Prompt on Windows) and move to the assignment folder.

A.2. Creating a Virtual Environment from YAML File

A YAML file is provided to help you reproduce the recommended Python environment. Suppose the provided file is named:

`environment.yml`

Then create the environment by running:

`conda env create -f environment.yml`

This will automatically install the required Python version and packages.

A.3. Activating the Environment

After creation, activate the environment using:

`conda activate cse30301`

(Use the actual environment name specified inside the YAML file.)

A.4. Running the Assignment Code

After activation, you may run:

`python evaluate_public.py`

`python plot_regret.py`

to test your implementation.